

EJERCICIO EVALUABLE 1: SISTEMAS DISTRIBUIDOS

CURSO 2025-2026

ANA CLAVER MIRANDA (100475965)

DAVID PAZ MONTOYA (100498667)

1. PARTE A: Versión no distribuida

1.1 Diseño del sistema

Para la primera parte de la práctica se ha desarrollado una versión **no distribuida** del servicio de almacenamiento de tuplas $\langle key, value1, value2, value3 \rangle$. Para esta parte del proyecto todas las operaciones se llevan a cabo sin comunicación entre procesos, puesto que todas las operaciones se realizan en uno solo. El sistema está formado por los siguientes ficheros:

- **claves.c**: Contiene la lógica del servicio y donde se implementan las operaciones que permiten gestionar las tuplas.
- **claves.h**: Define la interfaz del servicio y declara las funciones de la API (*destroy*, *set_value*, *get_value*, *modify_value*, *delete_key* y *exist*).
- **libclaves.so**: Permite que otros programas puedan utilizar estas funciones sin tener que incluir directamente su implementación.

Para comprobar que el servicio funciona correctamente se utiliza un programa cliente de ejemplo, **app-cliente.c**, que realiza llamadas a las funciones de la API y permite probar las distintas operaciones disponibles. De esta forma, el cliente solo interactúa con la biblioteca, mientras que toda la lógica interna del almacenamiento queda encapsulada dentro de ella.

1.2 Almacenamiento de las tuplas

Para el almacenamiento de las tuplas se ha utilizado una estructura de datos dinámica basada en una **lista enlazada**, puesto que simplifica la implementación y nos permite gestionar dinámicamente el número de tuplas. Sin embargo, esta elección también conlleva que las búsquedas tienen complejidad lineal $O(n)$, ya que es necesario recorrer la lista hasta encontrar la clave correspondiente. Cada elemento de la lista representa una tupla del sistema e incluye todos los campos asociados a una clave.

Esta estructura nos ha parecido la correcta porque permite añadir y eliminar elementos de forma flexible y, además, no impone un límite fijo al número de tuplas que se pueden almacenar. De esta manera, el sistema puede crecer dinámicamente, según sea necesario. Cada nodo de la lista guarda los siguientes elementos:

- la **clave (key)**, que identifica la tupla.
- el **valor value1**, que es una cadena de caracteres.
- el **valor value2**, que consiste en un vector de números de tipo float junto con su tamaño.
- el **valor value3**, que es una estructura con tres valores enteros (x, y, z).

Además, cada nodo incluye un puntero al siguiente elemento de la lista, permitiendo enlazar todas las tuplas almacenadas. Las operaciones de la API (*set_value*, *get_value*, *modify_value*, *delete_key* y *exist*) se implementan recorriendo esta estructura para localizar la clave correspondiente y realizar la operación solicitada. Por ejemplo, cuando se inserta una nueva tupla se añade un nuevo nodo a la lista, mientras que las operaciones de consulta o modificación buscan primero la clave antes de devolver o actualizar los datos.

1.3 Control de concurrencia y condiciones de carrera

Aunque esta versión no está distribuida, ya que se suelen ejecutar las operaciones de forma secuencial, la biblioteca se diseñó de forma segura para entornos multihilo, lo que permite reutilizar la misma implementación en el servidor concurrente de la versión distribuida. En estas situaciones pueden aparecer condiciones de carrera, que ocurren cuando varios hilos intentan acceder o modificar la misma información simultáneamente.

Para evitar este tipo de problemas se ha utilizado un **mutex** que protege el acceso a la estructura de datos donde se almacenan las tuplas. Antes de realizar cualquier operación sobre la lista, el hilo correspondiente bloquea el mutex. Una vez que la operación ha terminado, el mutex se libera para que otros hilos puedan acceder a la estructura.

2. PARTE B: Versión distribuida

Una vez desarrollada la versión local del servicio de almacenamiento de tuplas, el siguiente paso fue construir una versión distribuida que permitiera a distintos procesos cliente acceder al servicio a través de un servidor central. Para ello se adoptó una arquitectura **cliente-servidor**, donde el servidor mantiene el estado del sistema y los clientes interactúan con él mediante un mecanismo de comunicación basado en colas de mensajes *POSIX*.

En esta arquitectura el almacenamiento de los datos no cambia respecto a la versión anterior. El servidor sigue utilizando la biblioteca desarrollada en la Parte A (*libclaves.so*), que implementa todas las operaciones del servicio sobre una lista enlazada protegida mediante un mutex. La diferencia es que ahora únicamente el servidor puede acceder directamente a esta biblioteca.

Las aplicaciones cliente utilizan una biblioteca intermedia denominada **proxy**, implementada en el fichero *proxy-mq.c*. Esta biblioteca ofrece exactamente la misma interfaz definida en *claves.h*, por lo que desde el punto de vista del cliente el uso del servicio es idéntico tanto en la versión local como en la distribuida. Internamente, sin embargo, el *proxy* se encarga de transformar cada llamada de la API en un mensaje que se envía al servidor.

Para representar las peticiones y las respuestas se han definido dos estructuras de datos (*Request* y *Response*) en el fichero *comun.h*. Estas estructuras contienen todos los campos necesarios para representar una operación del servicio (tipo de operación, clave, valores asociados, etc.) y el resultado devuelto por el servidor.

De esta forma, cada operación solicitada por un cliente se transforma en un mensaje que viaja hasta el servidor, donde se ejecuta utilizando la implementación local del servicio.

2.1 Procesamiento concurrente de peticiones

Uno de los requisitos principales del sistema distribuido era que el servidor pudiera atender múltiples clientes simultáneamente. Para ello se diseñó un servidor concurrente utilizando **hilos bajo demanda**.

El servidor mantiene una cola de mensajes pública donde los clientes envían sus solicitudes. El proceso servidor entra en un bucle de ejecución en el que permanece bloqueado esperando nuevas peticiones mediante la llamada *mq_receive*. Cuando se recibe un mensaje válido, el servidor crea un nuevo hilo utilizando *pthread_create* para procesar esa solicitud concreta.

Cada uno de estos hilos ejecuta la función encargada de procesar la petición. Dentro de este hilo se analiza el tipo de operación solicitada y se invoca la función correspondiente de la biblioteca local implementada en la Parte A.

Una vez ejecutada la operación, el hilo construye la respuesta y la envía al cliente utilizando la cola de mensajes indicada en la solicitud. Tras enviar la respuesta, el hilo finaliza su ejecución.

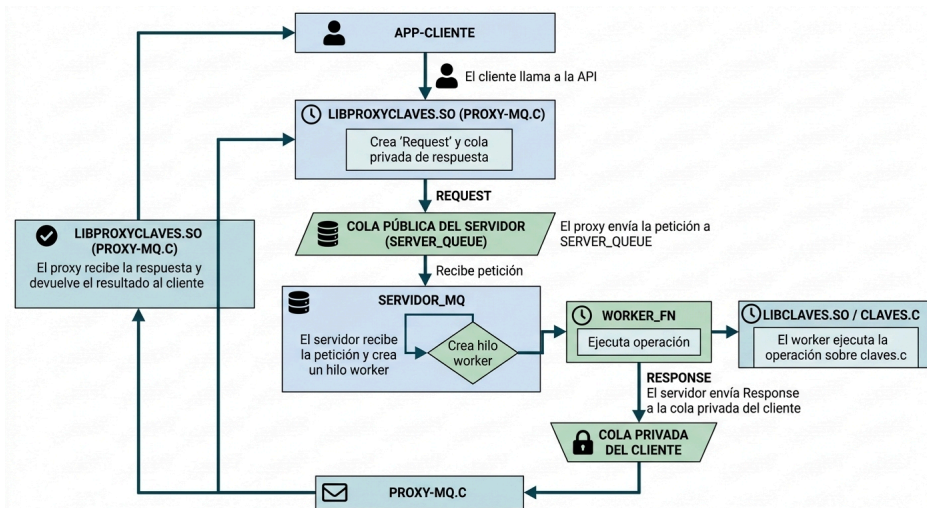
Los hilos se crean en modo *detached*, lo que significa que el sistema libera automáticamente sus recursos cuando terminan. Esto simplifica la gestión del servidor, ya que no es necesario utilizar *pthread_join* ni mantener estructuras adicionales para controlar los hilos activos.

Se eligió este modelo de concurrencia porque resulta sencillo de implementar y se adapta bien al tipo de carga del sistema, donde cada petición puede procesarse de forma independiente.

2.2 Protocolo de comunicación

La comunicación entre clientes y servidor se realiza mediante **colas de mensajes POSIX**, utilizando un esquema de interacción basado en peticiones y respuestas.

Cuando un cliente invoca una función de la API, el *proxy* crea una estructura *Request* que contiene todos los parámetros necesarios para ejecutar la operación solicitada. Además, el cliente genera dinámicamente el nombre de una cola de mensajes privada que se utilizará para recibir la respuesta del servidor. Para garantizar que cada cola sea única, el nombre se construye utilizando el identificador del proceso cliente (*PID*) y un contador interno.



El proceso de comunicación sigue los siguientes pasos:

1. El cliente crea una cola de mensajes privada para recibir la respuesta.
2. El cliente envía la estructura *Request* a la cola pública del servidor.
3. El servidor recibe la petición y crea un hilo para procesarla.
4. El hilo ejecuta la operación solicitada utilizando la biblioteca local.
5. El servidor envía una estructura *Response* a la cola privada del cliente.
6. El cliente recibe la respuesta y elimina su cola temporal.

Este diseño permite que múltiples clientes realicen operaciones de forma simultánea sin interferir entre sí, ya que cada petición utiliza una cola de respuesta independiente. Además, para evitar bloqueos indefinidos en caso de fallo del servidor, el cliente espera la respuesta utilizando la función *mq_timedreceive*, que establece un tiempo máximo de espera. Si el servidor no responde dentro de ese tiempo, el *proxy* interpreta la situación como un error de comunicación y devuelve un código de error al cliente.

3. COMPILACIÓN Y EJECUTABLE

3.1 Proceso de compilación: El proceso de compilación genera varios componentes del sistema.

Biblioteca local del servicio

La implementación local del servicio se encuentra en el fichero *claves.c*. Este archivo se compila como una biblioteca dinámica denominada *libclaves.so*; esta biblioteca contiene toda la lógica de almacenamiento de las tuplas clave-valor.

Cliente local

El programa *app-cliente.c* se compila enlazándolo con la biblioteca *libclaves.so*, generando el ejecutable *cliente_local*. Este programa permite ejecutar el servicio directamente sobre la implementación local y verificar el funcionamiento de la API sin utilizar comunicación entre procesos.

Biblioteca proxy del cliente

Para la versión distribuida se implementa un *proxy* cliente en el fichero *proxy-mq.c*. Este componente encapsula la comunicación con el servidor utilizando colas de mensajes *POSIX*. El *proxy* se compila como biblioteca dinámica (*libproxyclaves.so*) con el objetivo de mantener exactamente la misma interfaz que la biblioteca local.

Cliente distribuido

El mismo programa cliente, *app-cliente.c*, se reutiliza para probar la versión distribuida del servicio. En este caso se enlaza con la biblioteca *libproxyclaves.so*, generando el ejecutable *cliente_distribuido*. De esta forma el código del cliente no necesita modificarse: únicamente cambia la biblioteca utilizada para acceder al servicio.

Servidor

El servidor se implementa en el fichero *servidor-mq.c*. Este programa se compila como un ejecutable independiente denominado *servidor_mq*. Durante la compilación se enlaza con la biblioteca *libclaves.so*, que se utiliza internamente para ejecutar las operaciones solicitadas por los clientes.

El **Makefile** incluye también la regla *make clean*, que elimina los ficheros generados durante la compilación (objetos, bibliotecas y ejecutables), permitiendo recompilar el proyecto desde cero cuando sea necesario. Para compilar todos los componentes del sistema basta con ejecutar: *make*

3.2 Ejecución del sistema

Una vez compilado el proyecto es posible ejecutar tanto la versión local como la versión distribuida del servicio.

Pruebas de la versión local

Para comprobar el funcionamiento de la implementación local se puede ejecutar: `./cliente_local`

Este programa ejecuta una serie de operaciones sobre la biblioteca *libclaves.so*, permitiendo verificar que las funciones del servicio se comportan correctamente.

También se puede ejecutar el conjunto de pruebas automatizadas mediante: `./tests_local`

Pruebas de la versión distribuida

Para probar el sistema distribuido es necesario ejecutar primero el servidor: `./servidor_mq`

Al iniciarse, el servidor crea una cola de mensajes POSIX que utilizará para recibir solicitudes de los clientes y queda a la espera de nuevas peticiones.

En otra terminal se puede ejecutar el cliente distribuido: `./cliente_distribuido`

Este cliente utiliza el *proxy* para enviar las solicitudes al servidor y recibir las respuestas correspondientes. También se pueden ejecutar las pruebas automatizadas sobre la versión distribuida mediante: `./tests_distribuido`.

4. BATERÍA DE PRUEBAS Y SU EJECUCIÓN

4.1 Pruebas funcionales

```
[OK] delete_key empty key -> -1
[OK] exist empty key -> -1
[OK] set_value long key -> -1
[OK] get_value long key -> -1
[OK] modify_value long key -> -1
[OK] exist long key -> -1
[OK] delete_key long key -> -1
[OK] concurrency set_value same key -> 1 success, 9 duplicate fails
==== RESUMEN: OK=46 FAIL=0 ====
```

En primer lugar, hemos realizado pruebas funcionales para verificar que todas las operaciones definidas en la API (*set_value*, *get_value*, *modify_value*, *delete_key*, *exist* y *destroy*) funcionan correctamente.

Para ello hemos implementado el programa ***tests_local***, que ejecuta diferentes operaciones sobre la implementación local del servicio. Este programa comprueba tanto los códigos de retorno de cada

operación como la integridad de los datos almacenados y recuperados, incluyendo la cadena *value1*, el vector de floats y la estructura Paquete.

Posteriormente se ejecutaron las mismas pruebas utilizando la versión distribuida del sistema mediante el programa ***tests_distribuido***, que utiliza el *proxy* cliente para comunicarse con el servidor mediante colas de mensajes *POSIX*. Esto permite verificar que el comportamiento del servicio es idéntico desde el punto de vista del cliente en ambas implementaciones.

4.2 Tratamiento de errores

También se evaluó el comportamiento del sistema ante situaciones incorrectas o entradas inválidas (**-1**). Entre los casos probados se incluyen:

- inserción de claves duplicadas
- acceso a claves inexistentes
- valores de *N_value2* fuera del rango permitido
- cadenas de longitud superior al límite establecido
- uso de punteros nulos en parámetros obligatorios

En la versión distribuida se distingue además entre errores del servicio y errores de comunicación. Para comprobar este comportamiento se desarrolló el programa ***test_comunicacion***, que ejecuta una operación cuando el servidor no está activo. En este caso el cliente detecta el fallo de comunicación y devuelve el código **-2**.

4.3 Pruebas de concurrencia

Dado que el servidor procesa las solicitudes mediante hilos concurrentes, también se realizaron pruebas para comprobar el comportamiento del sistema en escenarios con múltiples clientes.

Estas pruebas verifican que las operaciones sobre la estructura de datos compartida se realizan de forma segura, gracias al uso del *mutex* en la implementación local. En particular, se comprobó que cuando varios hilos intentan insertar simultáneamente una clave idéntica, únicamente una inserción tiene éxito mientras que el resto fallan indicando que la clave ya existe.

5. TRABAJO EXTRA:

5.1 Benchmark: ejecución local vs distribuida

Para medir el coste introducido por la arquitectura distribuida se desarrolló un pequeño programa de *benchmark* que ejecuta **10000 operaciones consecutivas** sobre la API.

- **Modo local:** las llamadas a la API se realizan directamente sobre la biblioteca claves.
- **Modo distribuido:** las llamadas se realizan a través del *proxy*, utilizando colas de mensajes *POSIX* para comunicarse con el servidor.

Modo	Tiempo total (10000 ops)	Tiempo medio de operación
Local	0.20 ms	0.00002 ms
Distribuido	363.68 ms	0.03637 ms

Los resultados obtenidos muestran que el modo distribuido introduce una latencia adicional respecto a la versión local. Esta diferencia se debe principalmente al uso de colas de mensajes *POSIX* para la comunicación entre procesos, así como a la necesidad de serializar y deserializar las estructuras de datos intercambiadas. Además, influyen otros factores como la sincronización entre procesos y la creación y gestión de hilos en el servidor para atender múltiples peticiones concurrentes. Por tanto, el coste principal no proviene del procesamiento de la operación en sí, sino del mecanismo de comunicación entre procesos.

5.2 Prueba de concurrencia del servidor

```
aclaver@fedora:~/Documentos/Sistemas Distribuidos$ cd ~/D
Documentos/Sistemas Distribuidos/ej1
aclaver@fedora:~/Documentos/Sistemas Distribuidos/ej1$ ./
stress_test.sh
Lanzando 20 clientes concurrentes...
=== DEMO APP-CLIENTE ===

=== DEMO APP-CLIENTE ===

destroy(): 0
=== DEMO APP-CLIENTE ===

=== DEMO APP-CLIENTE ===

destroy(): 0
destroy(): 0
=== DEMO APP-CLIENTE ===
destroy(): 0
```

Con el fin de verificar que el servidor implementa correctamente un modelo concurrente, se ejecutó una prueba lanzando múltiples clientes simultáneamente mediante el script *stress_test.sh*. Durante la ejecución se imprimió en el servidor el identificador de hilo (*pthread_self()*), observándose que diferentes peticiones eran procesadas por hilos distintos. Esto confirma que el servidor utiliza un modelo de concurrencia basado en **creación de hilos bajo demanda**, permitiendo procesar múltiples peticiones de forma paralela.

5.3 Prueba de carga híbrida

Finalmente, se realizó una prueba de carga híbrida combinando **clientes locales y clientes distribuidos**. Para ello se implementó el script *hybrid_test.sh*, que lanza simultáneamente:

- **100 clientes locales**
- **50 clientes distribuidos**

Esta prueba permite evaluar el comportamiento del sistema cuando conviven diferentes tipos de acceso al API. Durante la ejecución se observó que el servidor seguía procesando correctamente múltiples peticiones concurrentes, manteniendo la estabilidad del sistema incluso bajo una carga mixta.

```
aclaver@fedora:~/Documentos/Sistemas Distribuidos$ cd ~/Documentos/Siste
mas Distribuidos/ej1
aclaver@fedora:~/Documentos/Sistemas Distribuidos/ej1$ ./hybrid_test.sh
Lanzando carga híbrida: 100 clientes locales + 50 distribuidos
Todos los clientes terminaron

.o claves.o -pthread -lrt
aclaver@fedora:~/Documentos/Sistemas Distribuidos/ej1$ ./
/servidor_mq
Servidor MQ escuchando en /mq_claves_srv_100475965_10049
8667
```

ACLARACIONES:

1. Las pruebas se han ejecutado tanto en *Guernika* como en local, desde cero.
2. Se ha usado Inteligencia Artificial como soporte de la memoria, para la generación del diagrama de flujo y la resolución de dudas específicas.